

# Mystery Room — Team 4

## Backend Study Guide

This guide explains the main concepts and implementation of the **Mystery Room — Team 4** backend using **Node.js + Express.js**.

The purpose is to understand how the backend is structured, how requests move through the application, and how the server controls the game's logic and state.

---

## 1. Project Overview

**Mystery Room** is a mystery / escape-room style application.

The player must solve several stages. Each stage contains a question, possible answers, a correct answer, a hint, and a clue.

The backend provides the API that connects the game logic with the frontend.

The backend is responsible for:

- Providing mystery information
- Providing the current question and options
- Checking submitted answers
- Moving the player to the next stage
- Detecting when the mystery is solved
- Providing hints
- Managing the current game state
- Returning appropriate HTTP responses

The frontend should mainly handle the user interface, while the backend should control the actual rules of the game.

---

## 2. Backend Architecture

The backend is divided into several parts:

```
server/  
├─ controllers/  
│   └─ mysteryController.js  
├─ data/  
│   └─ mysteries.js  
├─ routes/  
│   └─ mysteries.js  
├─ index.js  
├─ store.js  
├─ utils.js  
├─ package.json  
└─ .env
```

The architecture follows a simple separation of responsibilities:

```
Client  
  ↓  
Express Application  
  ↓  
Router  
  ↓  
Controller  
  ↓  
Store / Data  
  ↓  
Game Logic  
  ↓  
JSON Response
```

This structure prevents the entire backend from becoming one large file.

For example, the routes do not contain the answer-checking logic. They only connect an endpoint to the correct controller.

---

## 3. Node.js and Express.js

The project uses **Node.js** as the runtime environment.

Node.js allows JavaScript code to run on the server instead of only inside the browser.

The project uses **Express.js** to make creating an HTTP server and API easier.

The application starts with:

```
import express from "express";
```

```
const app = express();
```

`express()` creates the application object.

After creating it, we can add:

- Middleware
- Routes
- HTTP handlers

Express is therefore the main framework connecting HTTP requests to our backend logic.

---

## 4. package.json and ES Modules

The project uses:

```
{  
  "type": "module"  
}
```

This allows ES Module syntax:

```
import express from "express";
```

The main script is:

```
"scripts": {  
  "start": "node index.js"  
}
```

Therefore the backend can be started with:

```
npm start
```

The main dependencies are:

```
"dependencies": {  
  "dotenv": "^18.0.4",  
  "express": "^4.19.2"  
}
```

### Express

Used to create the HTTP server and API.

### dotenv

Used to load environment variables from `.env`.

The package configuration defines how the backend runs and which external libraries it requires.

---

## 5. Environment Variables

The project contains:

```
PORT=3001
```

This value is loaded using:

```
import dotenv from "dotenv";

dotenv.config();
```

Then:

```
const PORT = process.env.PORT || 3001;
```

This means the server first checks whether `PORT` exists in the environment.

If it does, that value is used.

Otherwise, the application falls back to:

```
3001
```

Environment variables separate configuration from application code.

For this project, the backend therefore runs by default at:

```
http://localhost:3001
```

---

## 6. Express Middleware

The backend uses middleware before reaching the routes.

The first important middleware is:

```
app.use(express.json());
```

This allows Express to parse JSON request bodies.

For example, when the frontend sends:

```
{
  "answer": "42"
}
```

the backend can access:

```
req.body.answer;
```

Without `express.json()`, the JSON body would not be automatically parsed.

---

The project also contains logging middleware:

```
app.use((req, res, next) => {  
  console.log(req.method, req.path);  
  next();  
});
```

For example:

```
GET /api/mysteries  
POST /api/mysteries/1/answers
```

The call to:

```
next();
```

allows the request to continue.

This demonstrates the basic middleware flow in Express:

```
Request  
↓  
Middleware  
↓  
Next middleware / Route  
↓  
Response
```

---

## 7. Main Server Setup

The main server is defined in `index.js`.

The important parts are:

```
import express from "express";
import mysteriesRouter from "../routes/mysteries.js";
import dotenv from "dotenv";

dotenv.config();

const PORT = process.env.PORT || 3001;

const app = express();

app.use(express.json());

app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});

app.use("/api/mysteries", mysteriesRouter);
```

Finally, the server starts with:

```
app.listen(PORT, () => {
  console.log(`Mystery Room API running at http://localhost:${PORT}`);
});
```

This file is responsible for assembling the backend.

It does not contain the actual mystery-solving logic.

That logic belongs in the controllers.

---

## 8. Routing and API Design

The mystery routes are defined in:

routes/mysteries.js

The router contains:

```
router.get("/", getAllMysteries);
router.get("/:id", getMysteryById);
router.get("/:id/clues", getCluesByMysteryId);
router.post("/:id/answers", submitAnswer);
router.patch("/:id/hint", requestHint);
```

Because the router is mounted at:

```
app.use("/api/mysteries", mysteriesRouter);
```

the complete API becomes:

| Method | Endpoint                   | Purpose           |
|--------|----------------------------|-------------------|
| GET    | /api/mysteries             | Get all mysteries |
| GET    | /api/mysteries/:id         | Get one mystery   |
| GET    | /api/mysteries/:id/clues   | Get clues         |
| POST   | /api/mysteries/:id/answers | Submit an answer  |
| PATCH  | /api/mysteries/:id/hint    | Request a hint    |

The route layer answers the question:

"Which controller should handle this request?"

The controller then decides what the application should actually do.

## 9. Mystery Data Structure

The initial mystery is stored in:

```
data/mysteries.js
```

The main structure is:

```
{
  id: 1,
  slug: "sealed-room",
  title: "الغرفة المغلقة في سوق الحميدية",
  intro: "...",
  totalStages: 3,

  stages: [
    {
      id: 0,
      question: "...",
      answer: "brass",
      options: ["brass", "iron", "gold", "silver"],
      hint: "...",
      clue: "..."
    }
  ],

  solved: false,
  currentStage: 0,
  hintsUsed: 0
}
```

The mystery contains two different types of information:

## Content

title  
intro  
question  
options  
answer  
hint  
clue

## Runtime state

solved  
currentStage  
hintsUsed

This distinction is important because content describes the game, while runtime state describes the current player's progress.

---

## 10. Stages and Game Progression

The current mystery contains three stages.

The answers are:

Stage 0 → brass

Stage 1 → 42

Stage 2 → 12

The player must solve them in order.

The backend stores:

```
currentStage;
```

to know which question the player is currently solving.

Initially:

```
currentStage: 0;
```

After solving stage 0:

```
0 → 1
```

After solving stage 1:

```
1 → 2
```

After solving the final stage, the mystery becomes:

```
solved: true;
```

This is why the backend is more than a simple data provider. It actually controls the progression of the game.

---

## 11. Store and In-Memory State

The controllers import:

```
import { mysteries, findMysteryById } from "../store.js";
```

The store provides access to the active mystery state.

The controller can therefore find a mystery by ID and update values such as:

```
mystery.currentStage;  
mystery.solved;  
mystery.hintsUsed;
```

The current implementation uses **in-memory state**.

This means the application keeps the state while the Node.js process is running.

For example:

```
Server starts  
  ↓  
Initial mystery loaded  
  ↓  
Player solves Stage 1  
  ↓  
currentStage becomes 1  
  ↓  
Server restarts  
  ↓  
Initial state is loaded again
```

So this implementation does not provide persistent storage.

A future version could replace this with a database such as SQLite, PostgreSQL, or MongoDB.

---

## 12. Controllers and Separation of Concerns

The main controller file is:

```
controllers/mysteryController.js
```

It contains:

```
getAllMysteries;  
getMysteryById;  
getCluesByMysteryId;  
submitAnswer;  
requestHint;
```

A controller receives:

```
req;  
res;
```

and performs the required logic.

For example:

```
export function getMysteryById(req, res) {  
  // ...  
}
```

The route decides **which controller runs**.

The controller decides **what should happen**.

This separation is one of the most important architectural ideas in the project.

---

## 13. Public Mystery Response

The controller contains:

```
function toPublicMystery(mystery) {
```

Its purpose is to transform the internal mystery object into the data that can safely be sent to the frontend.

For example, it returns:

```
return {  
  id: mystery.id,  
  slug: mystery.slug,  
  title: mystery.title,  
  intro: mystery.intro,  
  totalStages: mystery.totalStages,  
  currentStage: mystery.currentStage,  
  solved: mystery.solved,  
  hintsUsed: mystery.hintsUsed,  
  currentQuestion: currentStage ? currentStage.question : "",  
  currentOptions: currentStage ? currentStage.options : [],  
};
```

Notice that the correct answer is not included.

The backend therefore separates:

```
Internal Data  
  ↓  
Public Data
```

This is an important API design principle.

The frontend receives what it needs to display the game without receiving the solution in advance.

---

## 14. GET Endpoints

There are three GET operations.

### Get all mysteries

GET /api/mysteries

Implementation:

```
export function getAllMysteries(req, res) {  
  res.json(mysteries.map(toPublicMystery));  
}
```

Each mystery is converted into its public representation.

---

### Get one mystery

GET /api/mysteries/:id

The ID comes from:

```
req.params.id;
```

It is converted to a number:

```
const id = Number(req.params.id);
```

Then the store is searched.

If the mystery does not exist:

```
return res.status(404).json({  
  error: `No mystery found with id ${id}.`,  
});
```

Otherwise:

```
res.json(toPublicMystery(mystery));
```

---

## Get clues

GET /api/mysteries/:id/clues

The backend maps the stages into:

```
const clues = mystery.stages.map((stage) => ({
  stage: stage.id,
  text: stage.clue,
}));
```

This creates a simpler response containing only the stage number and clue text.

---

## 15. POST Answer Endpoint

The core game endpoint is:

POST /api/mysteries/:id/answers

The frontend sends:

```
{
  "answer": "42"
}
```

The backend processes the request in several steps:

1. Find the mystery  
↓
2. Validate the request body  
↓
3. Check whether the mystery is already solved  
↓
4. Get the current stage  
↓
5. Normalize the submitted answer  
↓
6. Compare it with the correct answer  
↓
7. Return failure or advance the game

This endpoint contains the most important business logic in the backend.

---

## 16. Input Validation

Before checking the answer, the controller verifies that it is valid:

```
if (
  !req.body ||
  typeof req.body.answer !== "string" ||
  req.body.answer.trim() === ""
) {
  return res.status(400).json({ error: "Answer must be a non-empty string." });
}
```

This prevents invalid input such as:

```
{}
```

or:

```
{
  "answer": 42
}
```

or:

```
{
  "answer": ""
}
```

The endpoint responds with:

```
400 Bad Request
```

This is important because the backend should not assume that everything received from the client is valid.

---

## 17. Answer Checking and Stage Progression

The current stage is retrieved using:

```
const currentStage = mystery.stages[mystery.currentStage];
```

The submitted answer is normalized:

```
const submitted = req.body.answer.trim().toLowerCase();
```

The correct answer is normalized too:

```
const correct = currentStage.answer.toLowerCase();
```

Then:

```
if (submitted !== correct)
```

determines whether the answer is incorrect.

For a wrong answer:

```
return res.json({
  correct: false,
  message: "That does not match the evidence. Try again.",
});
```

The stage does not change.

For a correct answer:

```
const nextStage = mystery.currentStage + 1;
```

If another stage exists:

```
mystery.currentStage = nextStage;
```

The backend then returns:

```
{
  correct: true,
  message: "The lock clicks open. You may continue.",
  nextStage: nextStage
}
```

---

## 18. Solving the Final Stage

The backend checks whether the player has reached the end:

```
if (nextStage >= mystery.totalStages) {
```

If this is the final stage:

```
mystery.solved = true;
```

The response is:

```
{
  correct: true,
  message:
    "The final lock clicks open. The mystery is solved!",
  nextStage: null
}
```

This creates a clear state transition:

```
Before:
currentStage = 2
solved = false
```

After final correct answer:

```
solved = true
```

The server therefore decides when the game is actually completed.

The frontend cannot simply declare the mystery solved on its own.

---

## 19. PATCH Hint Endpoint and HTTP Status Codes

The hint endpoint is:

```
PATCH /api/mysteries/:id/hint
```

The controller first verifies that the mystery exists and has not already been solved.

Then:

```
const currentStage = mystery.stages[mystery.currentStage];

mystery.hintsUsed += 1;
```

The remaining value is calculated using:

```
const hintsRemaining = Math.max(0, 3 - mystery.hintsUsed);
```

The response contains:

```
{
  hint: currentStage.hint,
  hintsRemaining: hintsRemaining
}
```

The hint belongs to the current stage.

## HTTP Status Codes

The backend uses meaningful status codes:

| Situation              | Status          |
|------------------------|-----------------|
| Successful request     | 200 OK          |
| Invalid input          | 400 Bad Request |
| Mystery not found      | 404 Not Found   |
| Mystery already solved | 409 Conflict    |

Example:

```
if (!mystery) {  
  return res.status(404).json({  
    error: `No mystery found with id ${id}.`,  
  });  
}
```

Using proper status codes allows the frontend to understand the result of the request without relying only on the text message.

## 20. Complete Backend Flow and What I Built

The most important thing to understand about this project is the complete flow:

```
React Frontend  
  ↓  
HTTP Request  
  ↓  
Express Middleware  
  ↓  
Router  
  ↓  
Controller  
  ↓  
Store / Data  
  ↓  
Game Logic  
  ↓  
JSON Response  
  ↓  
React Frontend
```

For example, submitting an answer:

```
graph TD
    A[User selects an answer] --> B[Frontend sends POST request]
    B --> C[Express receives request]
    C --> D[Router sends it to submitAnswer]
    D --> E[Controller validates the input]
    E --> F[Controller finds the mystery]
    F --> G[Current stage is retrieved]
    G --> H[Answer is normalized and compared]
    H --> I[Wrong → return failure  
Correct → move to next stage]
    I --> J[Final stage → solved = true]
    J --> K[JSON response]
    K --> L[Frontend updates the UI]
```

The backend therefore provides more than endpoints. It contains the **core rules and state management of the game**.

## Main concepts demonstrated by the project

- Node.js
- Express.js
- ES Modules
- Environment Variables
- Middleware
- Routing
- Controllers
- REST API
- HTTP Methods
- Request Parameters
- Request Body
- JSON Responses
- Input Validation
- HTTP Status Codes
- State Management
- Data Transformation
- Server-side Validation
- Separation of Concerns
- Promises / async concepts
- Client-Server Architecture

The backend can be summarized in one sentence:

**The frontend displays the game, while the backend controls the data, rules, validation, and progression of the game.**

The current implementation is intentionally simple and uses in-memory state, but its structure provides a good foundation for adding a database, authentication, testing, persistent game sessions, and deployment in a future version.